Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа № 1 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Матвиенко Е. Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 11.11.26

Москва, 2026

Постановка задачи

Вариант 10.

В текстовом файле записаны числа (каждое на новой строке). Родительский процесс запрашивает у пользователя имя файла, открывает его и создает дочерний процесс. Дочерний процесс считывает числа из файла (через перенаправление стандартного ввода), проверяет каждое число на простоту:

- Если число **составное**, дочерний процесс записывает его в стандартный поток вывода (который перенаправлен в неименованный канал `pipe1`).
- Если число **отрицательное или простое**, дочерний процесс и родительский процесс завершают работу.

Количество чисел может быть произвольным. Взаимодействие между процессами осуществляется через один канал (`pipe`) и перенаправление потоков ввода-вывода.

Общий метод и алгоритм решения

В данной лабораторной работе реализуется межпроцессное взаимодействие (IPC) в ОС Linux с использованием исключительно **низкоуровневых** системных вызовов без применения стандартных потоков ввода-вывода (`stdio.h`) и динамического выделения памяти.

Использованные системные вызовы:

- pid_t fork(void); — создает новый дочерний процесс путем дублирования родительского.
- int pipe(int *fd); — создает неименованный канал для однонаправленной передачи данных от дочернего процесса к родителю.
- int open(const char *pathname, int flags); — открывает файл для чтения.
- int dup2(int oldfd, int newfd); — перенаправляет стандартные потоки ввода/вывода на дескрипторы открытых файлов и каналов.
- int execv(const char *path, char *const argv[]); — заменяет образ текущего процесса новым исполняемым файлом дочерней программы (child).
- ssize_t read(int fd, void *buf, size_t count); и ssize_t write(int fd, const void *buf, size_t count); — чтение и запись данных через файловые дескрипторы каналов.
- int close(int fd); — закрывает файловый дескриптор после окончания работы.
- pid_t waitpid(pid_t pid, int *status, int options); — ожидает завершения дочернего процесса.
- void _exit(int status); — немедленно завершает процесс без очистки буферов.

Алгоритм работы программы:

1. Родительский процесс запрашивает у пользователя путь к текстовому файлу с числами и открывает его системным вызовом open().
2. Создается один неименованный канал (pipe1) для передачи результатов от дочернего процесса к родителю.

3. Вызывается fork(), после чего в дочернем процессе стандартный ввод (stdin) подменяется на дескриптор открытого файла, а стандартный вывод (stdout) — на запись в pipe1. Затем образ дочернего процесса заменяется через execv() на исполняемый файл child.
4. Дочерний процесс построчно читает числа из файла (через stdin) и выполняет их проверку на простоту в соответствии с Вариантом 10.
5. Составные числа записываются в стандартный вывод (попадая в pipe1), откуда их считывает родительский процесс и выводит на экран. При обнаружении отрицательного или простого числа дочерний процесс завершается (_exit(0)), закрывая канал, что служит сигналом для завершения работы родителя.

Код программы

Родительский процесс (`parent.c`)

```c
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

static char CHILD_PROGRAM_NAME[] = "child";

static void write_error(const char *msg, size_t len) {
    write(STDERR_FILENO, msg, len);
}

int main(void) {
    char progpath[1024];
    ssize_t len = readlink("/proc/self/exe", progpath,
sizeof(progpath) - 1);
    if (len == -1) {
        const char err[] = "Error reading
/proc/self/exe\n";
        write_error(err, sizeof(err) - 1);
        _exit(1);
    }

    progpath[len] = '\0';
    while (len > 0 && progpath[len] != '/') {
        --len;
    }
    progpath[len] = '\0';

    char filename[1024];
    int idx = 0;
    {
        const char prompt[] = "Enter filename: ";
        write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);

        char c;
        while (read(STDIN_FILENO, &c, 1) > 0) {
            if (c == '\n' || c == '\r') {
```

```
                break;
            }
            if (idx < (int)sizeof(filename) - 1) {
                filename[idx++] = c;
            }
        }
        filename[idx] = '\0';
    }

    if (idx == 0) {
        const char err[] = "Filename is empty\n";
        write_error(err, sizeof(err) - 1);
        _exit(1);
    }

    int file_fd = open(filename, O_RDONLY);
    if (file_fd == -1) {
        const char err[] = "Cannot open specified
file\n";
        write_error(err, sizeof(err) - 1);
        _exit(1);
    }

    int pipe1[2];
    if (pipe(pipe1) == -1) {
        const char err[] = "Pipe creation failed\n";
        write_error(err, sizeof(err) - 1);
        close(file_fd);
        _exit(1);
    }

    pid_t pid = fork();
    if (pid == -1) {
        const char err[] = "Fork failed\n";
        write_error(err, sizeof(err) - 1);
        close(file_fd);
```

```
        close(pipe1[0]);
        close(pipe1[1]);
        _exit(1);
    }

    if (pid == 0) {
        close(pipe1[0]);

        if (dup2(file_fd, STDIN_FILENO) == -1) {
            _exit(1);
        }
        close(file_fd);

        if (dup2(pipe1[1], STDOUT_FILENO) == -1) {
            _exit(1);
        }
        close(pipe1[1]);

        char path[1024];
        int p_len = 0;
        while (progpath[p_len] != '\0' && p_len < 512) {
            path[p_len] = progpath[p_len];
            p_len++;
        }
        path[p_len++] = '/';
        int c_len = 0;
        while (CHILD_PROGRAM_NAME[c_len] != '\0' && p_len
< 1023) {
            path[p_len++] = CHILD_PROGRAM_NAME[c_len++];
        }
        path[p_len] = '\0';

        char *const args[] = {CHILD_PROGRAM_NAME, NULL};
        execv(path, args);

        const char err[] = "Execv failed\n";
```

```
        write_error(err, sizeof(err) - 1);
        _exit(1);
    } else {
        close(file_fd);
        close(pipe1[1]);

        char buf[1024];
        ssize_t bytes;

        while ((bytes = read(pipe1[0], buf, sizeof(buf)))
> 0) {
            write(STDOUT_FILENO, buf, bytes);
        }
        close(pipe1[0]);

        waitpid(pid, NULL, 0);
    }

    return 0;
}
```

child.c

```c
    #include <unistd.h>

static int is_prime(long long n) {
    if (n <= 1) return 0;
    if (n <= 3) return 1;
    if (n % 2 == 0 || n % 3 == 0) return 0;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return 0;
    }
    return 1;
}

static void process_line(char *line, int line_len) {
    if (line_len <= 0) return;
    line[line_len] = '\0';

    long long num = 0;
    int negative = 0;
    int start = 0;
    if (line[0] == '-') {
        negative = 1;
        start = 1;
    }

    if (start >= line_len) return;

    int valid = 1;
    for (int j = start; j < line_len; ++j) {
        if (line[j] >= '0' && line[j] <= '9') {
            num = num * 10 + (line[j] - '0');
        } else {
            valid = 0;
            break;
        }
    }
```

```
    if (valid) {
        if (negative) {
            num = -num;
        }

        if (num <= 1 || is_prime(num)) {
            _exit(0);
        } else {
            char out_buf[64];
            long long temp = num;
            char rev[64];
            int r_len = 0;
            int len = 0;

            while (temp > 0) {
                rev[r_len++] = (char)('0' + (temp % 10));
                temp /= 10;
            }

            while (r_len > 0) {
                out_buf[len++] = rev[--r_len];
            }
            out_buf[len++] = '\n';
            write(STDOUT_FILENO, out_buf, len);
        }
    }
}

int main(void) {
    char buf[1024];
    char line[1024];
    int line_len = 0;
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf)))
> 0) {
```

```
        for (ssize_t i = 0; i < bytes; ++i) {
            char c = buf[i];
            if (c == '\n' || c == '\r') {
                process_line(line, line_len);
                line_len = 0;
            } else {
                if (line_len < (int)sizeof(line) - 1) {
                    line[line_len++] = c;
                }
            }
        }
    }

    if (line_len > 0) {
        process_line(line, line_len);
    }

    return 0;
}
```

Протокол работы программы

Тестирование:

```
Bash
$ ./os_labs
Enter filename: lab1/numbers.txt
4
6
```

Содержимое тестового файла lab1/numbers.txt:

```
Plaintext
4
6
7
8
9
```

(Число 7 является простым, поэтому при его обработке дочерний процесс завершает работу, а на экран выводятся только предшествующие составные числа).

Вывод

В ходе выполнения лабораторной работы были закреплены навыки межпроцессного взаимодействия (IPC) в операционных системах семейства Linux с использованием системных вызовов fork, pipe, dup2 и execv. Были изучены механизмы перенаправления стандартных потоков ввода-вывода (включая связывание файловых дескрипторов файлов и каналов) и синхронизации процессов. Основная трудность заключалась в правильной настройке файловых дескрипторов и своевременном закрытии неиспользуемых концов канала во избежание блокировок